Ders 4 Genişletilmiş Notu: Sıralı Örüntü Madenciliği ve Gözetimli Öğrenme

Ders 4 Genişletilmiş Notu: Sıralı Örüntü Madenciliği ve Gözetimli Öğrenme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Birliktelik Kurallarından Sıralı Örüntülere
- 2. Kavramlar ve Gösterim
- 3. Veri Formatı: İşlemlerden Sequence Veritabanına
- 4. Sık Sequence Bulma Örneği
- 5. GSP Algoritması
- 6. Makine Öğrenmesi Algoritmalarına Genel Bakış
- 7. Gözetimli Öğrenmenin Temelleri
- 8. Karar Ağacı
- 9. Kural Tabanlı Sınıflandırma ve Aşırı Öğrenme
- 10. Naive Bayes
- 11. K En Yakın Komşu
- 12. Test Verisi Seçimi ve Çapraz Doğrulama

1. Birliktelik Kurallarından Sıralı Örüntülere

Geçen hafta kural madenciliğine (rule mining) girilmiş, birliktelik kuralı madenciliği (association rule mining) algoritmaları konuşulmuştu. Bu hafta konu sıralı örüntü madenciliği (sequential pattern mining). Hoca, kitaplarda "sıralı kural madenciliği" adıyla da geçtiğini söyledi. Yöntem, özellikle farklı log dosyalarında insanların gerçekleştirdiği popüler sıralı aksiyonları tespit etmek için kullanılıyor.

1.1 Market Sepeti Analizini Geliştirmek

Yöntemin ilk çıkış amacı, market sepeti verisi analizini (market basket data analysis) daha akılcı ve daha başarılı hale getirmektir. Market sepeti analizinde popüler olarak birlikte satılan ürünleri bulmak istiyorduk. A, B ve C ürünleri çok sayıda kişiye birlikte satılmışsa, A ürününü almış bir kişiye B ve C ürünlerini önerebiliyorduk.

Oysa elimizde satış sırası bilgisi de var. Alım satım işlemleri (transaction), yapıldıkları anlarla birlikte işlem veritabanına kaydedilir. Hocanın örneğiyle, aynı gün markete sabah, öğlen ve akşam olmak üzere üç kez gittiğinizi düşünün. Üç farklı işlem oluşturdunuz ve her gidişte farklı ürünler aldınız. Bu ürünleri sıralı alınmış ürünler olarak listeleyebiliriz.

Çok sayıda insanın önce A, arkasından B, sonra C aldığını fark edersek, elimizde popüler bir sıralı örüntü var demektir. Buna dayanarak öneri yapabiliriz: A ürününü almış kişiye önce B'yi, onu aldıktan sonra da C'yi öneririz.

Temel fark şudur: Birliktelik kuralları işlemler içindeki sırayı dikkate almaz. Sıralı örüntü madenciliği ise ürünlerin alım zamanını ve sırasını dikkate alarak popüler birliktelikleri ortaya çıkarır.

1.2 Web Madenciliğindeki Kullanımı

Hoca bu tekniğin web madenciliğinde çok kullanıldığını söyledi. Web sayfalarıyla etkileşim sırasında farklı tipte log dosyaları oluşur:

- **Uygulama sunucusunun logları:** Web uygulaması IIS veya Tomcat gibi bir sunucu üzerinde çalışıyorsa, sunucunun kendisinin oluşturduğu log dosyalarında veri toplanır. Bu veriler analiz edilerek insanların ne yaptığı anlaşılır ve sıralı gerçekleştirdikleri popüler aktiviteler tespit edilir.
- **Uygulamanın kendi logları:** Uygulamanın da ayrı bir log dosyası olabilir. Bu dosya ayrıştırılarak (parse) kullanıcıların o uygulama üzerindeki sıralı aktiviteleri bulunabilir.

1.3 Müşteri Terki ve "Next Best Action"

İkinci kullanım alanı, müşteri hareketlerini modelleyip başarılı aktivitelerin sıralı aksiyon zincirlerini bulmaktır. Bankacılık ve telekom sektörlerinde terk etme davranışını modellemek önemli bir problemdir. Buna terk analizi (churn analysis) denir. İnsanların terk edeceği tespit edildikten sonra onları sistemde tutmak için uygulanan yöntemlere de tutundurma modelleri (retention models) denir.

Pazarlama birimi bu amaçla bir aksiyon zinciri uygular. Önce kişiye telefon açılır ve bir teklif (offer) yapılır: Mevcut paketin ücret alınmadan yükseltileceği söylenir. Müşteri bir iş yeriyse arkasından ziyarete gelineceği söylenir. Terk etme davranışı gösteren müşterilere uygulanan zincirlerden popüler olanlar tespit edilirse, bir sonraki böyle müşteride nasıl ilerleneceği konusunda pazarlama birimine bilgi verilebilir. Buna "next best action" deniyor. Hoca bunu sıralı örüntü madenciliğinin terk analizindeki uygulanma şekli olarak gösterdi.

2. Kavramlar ve Gösterim

Algoritmalara geçmeden önce gösterime bakıldı. Birliktelik kurallarında olduğu gibi elimizde $i_1, i_2, \dots, i_m$ gibi öğeler (items) var. Öğe, bir markette satılan ürün olabileceği gibi, terk analizi örneğindeki gibi bir aksiyon da olabilir.

2.1 Sequence, Eleman ve Öğe Kümesi

Bir sequence s ile gösterilir ve şöyle yazılır:

$$s = \langle a_1 a_2 \dots a_r \rangle$$

Buradaki her a_i, sequence'ın bir elemanıdır (element) ve bir öğe kümesidir (itemset). Öğe kümesi tanımı bir önceki dersten biliniyor: İçinde bir ya da birden fazla öğe bulunabilir. Örneğin a_1 yalnızca $\{i_1\}$ olabileceği gibi $\{i_1, i_2\}$ de olabilir. a_2 yalnızca $\{i_3\}$ ya da $\{i_3, i_4, i_5\}$ olabilir. Sıralı örüntü madenciliği algoritmalarının bulmaya çalıştığı şey, bu sequence'lardan popüler olanlardır: sık sequence'lar (frequent sequences).

2.2 Size, Length ve k-Sequence

Bir sequence için iki ayrı büyüklük tanımlıdır:

- **Size:** Sequence içindeki eleman sayısı. $\langle a_1 a_2 \dots a_r \rangle$ için size r'dir.
- **Length:** Sequence içindeki toplam öğe sayısı. Yani bütün a_i'lerdeki öğeler toplandığında bulunan sayı.

Öğe kümelerinde k-itemset diyorduk. 3-itemset, içinde üç öğe olan kümeydi. Sequence için de benzer şekilde k-sequence denir. Hoca önce k'nın eleman sayısını gösterdiğini söyledi, sonra kendisi düzeltti: k, length'i gösterir. Yani bir sequence, içindeki toplam öğe sayısıyla adlandırılır.

Slayttaki örnek şuydu:

$$\langle \{3\}\{4,5\}\{8\} \rangle$$

- Bu sequence'ta 3 eleman var, dolayısıyla size 3'tür.
- Öğeler 3, 4, 5 ve 8'dir. Dört öğe olduğu için length 4'tür.
- Length kullanılarak adlandırılır: Bu bir 4-sequence'tır.

> **Ders sonrası not:** Hoca derste "eleman" sözcüğünü zaman zaman öğe anlamında da kullandı. Örneğin 4-sequence için "4 eleman içeren sequence", GSP anlatımında "eleman sayısı 2 olan sequence" dedi. Kendi tanımına göre doğrusu "4 öğe içeren" ve "öğe sayısı 2 olan"dır. Eleman yalnız süslü parantezlerle ayrılan kümeyi gösterir.

Soru: Length hesaplanırken tekrar eden öğeler de sayılıyor mu, yoksa benzersiz öğeler mi sayılıyor?

Cevap: Hoca length'in benzersiz (unique) hesaplandığını, içindeki öğelere bakıldığını söyledi. Size için ise eleman sayısına bakılır. Süslü parantez içindeki her şey sequence'ın bir elemanıdır, her eleman da kendi içinde bir kümedir.

> **Ders sonrası not:** Hoca length'in "unique" sayıldığını söyledi; doğrusu büyük olasılıkla, aynı öğe farklı elemanlarda geçiyorsa her geçişin ayrı sayılmasıdır. Yani $\langle\{3\}\{3,8\}\rangle$ gösteriminin length'i 3 olur. Tek bir eleman küme olduğu için aynı öğeyi iki kez içeremez, dolayısıyla tekrar yalnız farklı elemanlar arasında söz konusudur. Derste kullanılan örneklerde tekrar eden öğe olmadığı için sonuçlar bundan etkilenmez.

2.3 Alt Sequence ve Süper Sequence

Bir sequence, başka bir sequence'ın alt sequence'ı (subsequence) olabilir. Onu içeren sequence'a da süper sequence (super-sequence) denir. Bunun gerçekleşmesi için alt sequence'taki her elemanı kapsayan bir elemanın süper sequence'ta bulunması gerekir.

Hocanın örneği:

$$s_1 = \langle\{3\}\{4,5\}\{8\}\rangle, \qquad s_2 = \langle\{6\}\{3,7\}\{9\}\{4,5,8\}\{3,8\}\rangle$$

s_1'in elemanları sırayla kontrol edilir:

- $\{3\}$'ü kapsayan bir eleman var: $\{3,7\}$.
- $\{4,5\}$'i kapsayan bir eleman var: $\{4,5,8\}$.
- $\{8\}$'i kapsayan bir eleman var: $\{3,8\}$.

Her eleman için kapsayan bir eleman bulunduğundan s_1, s_2'nin alt sequence'ıdır. Yani "s_1, s_2'nin içinde vardır" denebilir.

Soru: Süper sequence'ta $\{4,5,8\}$ yerine 4, 5 ve 8 ayrı ayrı elemanlar olsaydı, s_2 yine s_1'i kapsar mıydı?

Cevap: Hayır. $\{4,5\}$ elemanını tek başına içeren bir küme mutlaka bulunmalıdır. Hoca, sorunun tersi durumu da açıkladı: 4 ile 5, s_1'de ayrı ayrı iki eleman olsaydı, bu kez onlar için iki ayrı kapsayan küme aranırdı.

Biçimsel tanımı hoca slayttan okudu: $s_1 = \langle a_1 \dots a_r\rangle$ dizisinin s_2'nin alt sequence'ı olması için a_1'i kapsayan, a_2'yi kapsayan, ..., a_r'yi kapsayan elemanlar s_2'de bulunmalıdır. O zaman "s_2, s_1'i içerir" denir.

> **Ders sonrası not:** Hoca anlatımda kapsayan elemanların sırasını açıkça söylemedi ve "s_1 kümesi s_2 kümesinin alt kümesi" ifadesini kullandı. Doğrusu büyük olasılıkla şudur: Sequence küme değildir ve sıra korunmalıdır. $s_2 = \langle b_1 \dots b_v\rangle$ için $a_1 \subseteq b_{j_1}, \dots, a_r \subseteq b_{j_r}$ olacak şekilde $j_1 < j_2 < \dots < j_r$ indisleri bulunmalıdır. Hocanın örneğinde indisler 2, 4 ve 5'tir, yani koşul sağlanır. Sıralı örüntünün birliktelik kuralından farkı da zaten budur.

3. Veri Formatı: İşlemlerden Sequence Veritabanına

Veri kümesi müşteri (customer) merkezli düzenlenmiştir. Her müşteri farklı zamanlarda gelmiştir ve her gelişinde aldığı ürünler görülür. Bu ürünler alım satım sırasında fişe düşer. Ör-

neğin bir fişe önce 40, sonra 60, sonra 70 düşmüşse, kasiyer ürünleri bu sırayla işlemiştir. Bunlar aynı işlemde birlikte alınmış öğeler olarak düşünülebilir.

Burada zamanın çözünürlüğü önemlidir. Örnek veride işlem zamanı gün olarak verilmiştir. Bu durumda 40 sabah, 60 öğlen, 70 akşam alınmış ve ayrı işlemlerde gerçekleşmiş olabilir, ama aynı gün olduğu için aynı elemana girerler. İşlem zamanı anlık olsaydı, bu üç ürün aynı anda alınmış sayılacak, aralarındaki sıra ise kasiyerin kasaya işleme sırasıyla belirlenecekti. Hocanın vardığı sonuç şu: Marketten yapılan bütün alışverişlerin ürün bazında bir zaman damgası ve bir sırası vardır. Bu sıraya bağlı olarak işlem veritabanı oluşturulabilir.

> **Ders sonrası not:** Hoca gün bazındaki zaman için "rezolüsyonu biraz yüksek" dedi; bağlamdan anlaşıldığı üzere kastedilen zaman ayrıntısının kaba, yani çözünürlüğün düşük olmasıdır.

İşlem veritabanından yola çıkılarak her müşteri için bir sequence yazılır. Aynı müşterinin her işlem zamanı sequence'ın bir elemanı olur:

- Müşteri 1: $\langle\{30\}\{90\}\rangle$. Önce 30'u, arkasından 90'ı almıştır.
- Müşteri 2: $\langle\{10, 20\}\{30\}\{40, 60, 70\}\rangle$. Üç elemanlı bir sequence.
- Müşteri 3: $\langle\{30, 50, 70\}\rangle$. Tek elemanlı, ama bu elemanda üç öğe var.
- Müşteri 4: $\langle\{30\}\{40, 70\}\{90\}\rangle$. Üç elemanlı.
- Müşteri 5: $\langle\{90\}\rangle$. Tek elemanlı.

Sıralı kural madenciliği algoritmaları, işlem verisinden dönüştürülmüş bu veri kümesi üzerinde çalışır.

4. Sık Sequence Bulma Örneği

Önceki derste sık öğe kümelerini (frequent itemsets) bulup onlardan birliktelik kuralları çıkarmıştık. Burada da sık sequence'lar bulunacak, sonra onlardan kurallar çıkarılacak. Bunun için verilen destek (support) değeri kullanılır: Belirli sayıda işlemde ya da müşteride geçmiş sequence'lar aranır.

Verilen: 5 müşteri ve %25 minimum destek. $0{,}25 \times 5 = 1{,}25$ olduğu için bir sequence'ın en az 2 müşteride geçmesi gerekir.

> **Ders sonrası not:** Hoca bu koşulu bir yerde "iki tanenin üzerinde", sonra "en az iki" diye söyledi. Doğrusu "en az 2 müşteri"dir, çünkü 1,25'ten büyük ilk tam sayı 2'dir. Hocanın yazdığı sonuçlar da bu koşula uyuyor.

Bir sequence bir müşteride tek başına durmasa bile, o müşterinin sequence'ının alt sequence'ıysa o müşteride geçiyor sayılır.

> **Ek bilgi (derste anlatılmadı):** Destek, müşteri sayısı üzerinden sayılır. Bir örüntü aynı müşterinin sequence'ında birden fazla yerde eşleşse bile o müşteri bir kez sayılır. Bu yüzden destek, eşleşme sayısı değil, örüntüyü içeren müşteri sayısının toplam müşteri sayısına oranıdır.

4.1 Length Değeri 1 Olan Sık Sequence'lar

Hoca her öğe için gereken iki müşteriyi göstererek ilerledi. Tam sayımlar şöyledir (ara adım eklendi):

Sequence	Geçtiği müşteriler	Sayı	Sık mı?
$\langle\{30\}\rangle$	1, 2, 3, 4	4	Evet
$\langle\{40\}\rangle$	2, 4	2	Evet
$\langle\{70\}\rangle$	2, 3, 4	3	Evet
$\langle\{90\}\rangle$	1, 4, 5	3	Evet

Hoca, 40 ve 70'in hiçbir müşteride tek başına bir eleman olmadığına dikkat çekti. Başka süper sequence'ların alt sequence'ı oldukları için geçmiş sayılırlar. 90 ise üç farklı müşteride görülür.

4.2 Length Değeri 2 Olan Sık Sequence'lar

Bu satırdaki her sequence'ın içinde iki öğe bulunmalı ve en az iki müşteride geçmelidir.

- $\langle\{30\}\{40\}\rangle$: Müşteri 2'de 30'dan sonra 40 var. Müşteri 4'te de 30'dan sonra 40 var. İki müşteri.
- $\langle\{30\}\{70\}\rangle$: Müşteri 2 ve 4'te 30'dan sonra 70 var. Müşteri 4'te 70, $\{40, 70\}$ elemanının içindedir, yani $\{70\}$ onun alt kümesi olarak sayılır.
- $\langle\{30\}\{90\}\rangle$: Müşteri 1 ve 4'te 30'dan sonra 90 var.
- $\langle\{40, 70\}\rangle$: Müşteri 2 ve 4'te 40 ile 70 aynı elemanın içindedir.

Hoca, 40 ile 70'in aynı süslü parantezde yazılmasının sebebini vurguladı: İkisi her iki müşteride de aynı elemandadır. 30 ve 90 da iki müşteride aynı elemanda olsaydı $\langle\{30, 90\}\rangle$ diye yazılacaktı. Ama iki müşteride de ayrı elemanlarda ve sıralı geçtikleri için $\langle\{30\}\{90\}\rangle$ yazılır.

Soru: 2-sequence belirlenirken size'a göre mi bakıyoruz, yoksa başka bir şeye göre mi?

Cevap: Öğe sayısına, yani length'e göre bakıyoruz. $\langle\{30\}\{40\}\rangle$'ın öğe sayısı 2, eleman sayısı 2'dir. $\langle\{40, 70\}\rangle$'ın öğe sayısı 2, eleman sayısı 1'dir. Aranan şey, içinde iki öğe bulunan ve en az iki müşteride geçen sequence'tır. Eleman sayısının önemi yoktur.

4.3 Length Değeri 3 Olan Sık Sequence

$\langle\{30\}\{40, 70\}\rangle$ tek sonuçtur. $\{40, 70\}$, Müşteri 2'de $\{40, 60, 70\}$ elemanının alt kümesidir ve 30'dan sonra gelir. Müşteri 4'te de 30'dan sonra $\{40, 70\}$ gelir. İki müşteride geçtiği için bu 3-sequence sıktır. Python ile bütün olası sequence'lar sayıldığında length değeri 4 olan sık sequence çıkmadı, dolayısıyla daha uzunu da yoktur.

4.4 Sonuç

Length	Sık sequence'lar
1	$\langle\{30\}\rangle$, $\langle\{40\}\rangle$, $\langle\{70\}\rangle$, $\langle\{90\}\rangle$
2	$\langle\{30\}\{40\}\rangle$, $\langle\{30\}\{70\}\rangle$, $\langle\{30\}\{90\}\rangle$, $\langle\{40, 70\}\rangle$
3	$\langle\{30\}\{40, 70\}\rangle$

5. GSP Algoritması

Algoritma, geçen hafta konuşulan Apriori'nin bir benzeridir ve yine aşağıdan yukarı (bottom-up) çalışır. Mantık aynıdır, ama birlikteliklere değil sıralı örüntülere bakılır. Adımlar:

1. Veritabanı bir kez okunur ve sık 1-sequence'lar bulunur.
2. `for` döngüsüne girilir. Sık 1-sequence'lar aday üretme fonksiyonuna (candidate-gen) verilir ve 2-sequence adayları elde edilir.
3. Veritabanı yeniden okunur. Aday sequence'ların veritabanında görülme sayılarına bakılır.
4. Adaylar içinden minimum destek değerini sağlayanlar seçilir. Böylece sık 2-sequence'lar, yani önceki örnekteki ikinci satır bulunur.
5. Döngünün başına dönülür. Bu kez sık 2-sequence'lardan 3-sequence adayları üretilir, veritabanı tekrar okunur ve böyle devam edilir.

Bu algoritmanın adı GSP'dir ve sıralı örüntü madenciliği algoritmalarından yalnızca biridir. Hoca bunun dışında Spark'ın makine öğrenmesi kütüphanesi bağlamında PrefixSpan ve SPADE algoritmalarını andı. Bu alan oldukça popüler ve aktif. Algoritmanın çok sayıda türevi üretilmiş ve her birinin farklı avantajları var.

> **Ek bilgi (derste anlatılmadı):** Apriori'de olduğu gibi GSP de aday sayısını azaltmak için aşağı doğru kapanma (downward closure) özelliğini kullanır. Bir sequence sıksa bütün alt sequence'ları da sıktır. Bu yüzden alt sequence'larından biri sık olmayan aday, veritabanı taranmadan elenebilir.

6. Makine Öğrenmesi Algoritmalarına Genel Bakış

Aradan sonra makine öğrenmesine geçildi. Hoca bu bölümü bir genel tekrar (overview) olarak düşünmemizi istedi. Çoğu öğrenci bu algoritmalarla daha önce karşılaştığı için ayrıntıya fazla inilmeyecek, web madenciliğinde kullanılan algoritmalardan bahsedilecek.

Web madenciliğinde amaç, web verisi üzerinden insanların davranışlarını modellemek ve web uygulamaları üzerinde bir sonra yapacakları hareketleri tahmin etmektir. Tahminler şu türden olabilir: Kişi belli bir ürünü alacak mı, belli bir ürünle ilgilenecek mi, sistemi terk edecek mi? Üzerinde çalışılan algoritmalar genellikle tıklama akışı (clickstream) verisinden çıkarılmış öz nitelikler üzerinde çalışır.

6.1 Gözetimli Öğrenme

Gözetimli öğrenme (supervised learning), Türkçede denetimli öğrenme diye de çevriliyor. İki başlığa ayrılır:

- **Sınıflandırma (classification):** Etiket kategorik, yani ayrık (discrete) bir değerdir. Sayısal değildir. Örneğin terk analizinde tıklama akışı verisinden müşterinin sistemi terk edip etmeyeceği tahmin edilir. Hocanın örneğiyle, Trendyol gibi bir e-ticaret sitesinde müşterinin giriş (login) ve çıkış (logout) arasındaki hareketlerinden öz nitelikler çıkarılıp model kurulur. Bu modelle müşterinin bir daha gelip gelmeyeceği veya ürün satın alıp almayacağı tahmin edilir.
- **Regresyon (regression):** Sayısal bir değer tahmin edilir. Özellikle finansal zaman serileri ve zaman serisi verilerinde kullanılır. Hocanın örneği kredi limitidir: Size 5 bin liralık bir limit tanımlandığında bunun gerçekte ne kadarını kullanacağınızı bankalar bilmek ister. Banka bunu regresyonla tahmin edebilirse, kredi önerilerini limiti daha yüksek oranda kullanacak müşterilere yapabilir.

6.2 Gözetimsiz Öğrenme

Gözetimsiz öğrenmede (unsupervised learning) verilerin etiketi yoktur. Etiketsiz veri üzerinden verinin yapısı hakkında bilgi edinilmek istenir. Üç grup sayıldı:

- **Kümeleme (clustering):** Birbirine yakın elemanlar kümelere ayrılır.
- **Birliktelik tabanlı yöntemler:** Önceki bölümde görülen birliktelik kuralı madenciliği ve sıralı kural madenciliği algoritmalarının hepsi bu gruptadır.
- **Boyut indirgeme (dimension reduction) ile genelleştirme:** Verinin boyutu küçültülerek genelleştirme yapılır.

6.3 Boyut İndirgeme ve PCA

Bu grubun örneği PCA algoritmasıdır. Diyelim ki 100 öz niteliğiniz var. PCA ile bu sayıyı 5'e veya 10'a indirirsiniz. Algoritma, sizin adınıza sanal öz nitelikler oluşturur. Kaç öz nitelik istediğiniz de algoritmanın girdilerinden biridir.

Kümelemeden farkı nedir? Hocaya göre boyut indirgeme, veriyi bir sonraki işlem için farklı bir biçime sokar. Verinin içindeki yapıyı henüz ortaya çıkarmaz, ama sonraki aşamada kullanılacak gözetimli öğrenme algoritmaları için veriyi daha öğrenilebilir hale getirir. Kümeleme ise hangi veri elemanlarının birbirine yakın olduğunu, yani yapının kendisini ortaya çıkarır. Bu nedenle boyut indirgeme algoritmaları genellikle kümeleme ve sınıflandırma algoritmalarından önce kullanılır.

Soru: Makine öğrenmesi için öz nitelik çıkarımında (feature extraction) çok sayıda öz nitelik çıkıyor. Etiketle korelasyonu düşük olanları elememiz gerekiyor. Bunu otomatik yapan kısım bu mu?

Cevap: Hoca "evet, aynen" dedi ve PCA algoritmasının bu grupta olduğunu söyledi.

Hoca ardından PCA tabanlı anomali tespiti (anomaly detection) örneğini anlattı. Web madenciliğinde çalışıldığı için oturum (session) verisi üzerinden gidilir. Öz nitelikler şunlar olabilir:

- Kişinin son 6 aydaki oturumlarında ortalama kaç sayfa görüntülediği (page view).
- Son 6 aydaki oturumlarında kaç tıklama yaptığı.

- Kaç kez ödeme (check-out) işlemi yaptığı.

Bunlar bir araya gelince öz nitelik vektörü oluşur. Her vektörün sonunda "anomalidir" veya "değildir" etiketi olduğunu, bu etiketleri iş analistlerinin önceden verdiğini ve elimizde bin tane böyle kayıt bulunduğunu düşünün. Bu öz niteliklerden hangilerinin birbiriyle daha anlamlı olduğunu, hangilerinin ayrıştırıcı gücü olduğunu bilmiyoruz. Boyut indirgeme algoritmaları bu öz nitelikleri temsil eden sanal öz nitelikler oluşturur. Birbiriyle ilişkili (correlated) öz nitelikleri birleştirir ve sayısı daha az, ayrıştırıcı gücü daha yüksek öz nitelikler verir. Böylece veri genelleştirilir ve üzerinde sınıflandırma algoritmaları kullanılabilir.

> **Ders sonrası not:** Hoca, PCA'nın etiketle ilişkisi düşük öz nitelikleri eleyen kısım olduğu sorusuna "evet" dedi ve ayrıştırıcı gücü en yüksek öz nitelikleri ürettiğini söyledi. Doğrusu büyük olasılıkla şudur: PCA gözetimsizdir, etiketi hiç kullanmaz. Öz niteliklerin doğrusal birleşimlerinden, verideki varyansı en iyi koruyan bileşenleri üretir. Yüksek varyans, sınıfları ayırma gücünün yüksek olduğunu garanti etmez. Etikete göre öz nitelik elemek ise öz nitelik seçimi (feature selection) yöntemlerinin işidir.

6.4 Sığ ve Derin Modeller

Hoca bir taksonomi slaydı gösterdi. Şimdiye kadar konuşulan gözetimli ve gözetimsiz yöntemler, sığ modeller (shallow models) altında toplanabilir. Yani veriyi analiz ederken derin öğrenme yöntemleri kullanılmaz. İki grup arasındaki fark hocaya göre şöyledir:

- **Sığ modeller:** Daha az veriyle beslenir. Verinin çeşitliliği derin öğrenmeye göre çok fazla değildir. Kullanılan veri kümeleri çok büyük değildir.
- **Derin öğrenme (deep learning):** Katman sayısı fazladır. Çok miktarda ve çeşitli, gerçekleşen hareketleri temsil eden veriyle beslenir.

Birliktelik kuralı madenciliği ve boyut indirgeme slaytta gösterilmiyordu, ama onlar da sığ modellerin altında yer alıyor. Derin öğrenmede de gözetimli ve gözetimsiz çeşitler var. Gözetimli tarafta yaygın kullanılan model, tekrarlayan sinir ağıdır (recurrent neural network, RNN). RNN'ler içinde de en yaygın kullanılan LSTM'dir.

> **Ders sonrası not:** Hoca RNN ve LSTM'in görüntü işleme ve görüntüde nesne tanımada çok başarılı olduğunu söyledi; doğrusu büyük olasılıkla görüntü ve nesne tanımada asıl başarılı olan modelin evrişimli sinir ağı (convolutional neural network, CNN) olmasıdır. RNN ve LSTM daha çok metin, konuşma ve zaman serisi gibi dizi verilerinde kullanılır.

Bu ders kapsamında öncelik gözetimli ve gözetimsiz yöntemlerin web madenciliğindeki uygulamalarıdır. Zaman kalırsa derin öğrenme algoritmalarından web madenciliğinde kullanılanlara da girilecek.

7. Gözetimli Öğrenmenin Temelleri

7.1 Örnekler ve Terimler

Hoca hızlı bir tekrar için örneklerle başladı.

Acil servis örneği: 17 hastanın bulunduğu bir acil serviste (emergency room) bu hastalarla ilgili veri var. Yeni gelen hastaları, verdikleri bilgilere dayanarak yüksek riskli veya düşük riskli diye etiketlemek istiyoruz. Yatak sayısı sınırlı olduğu için yeni gelen kişiyi alıp almayacağımıza buna göre karar vereceğiz. Daha önce acil bakıma alınmış hastaların hem verileri hem etiketleri elimizde olduğu için bu bir gözetimli öğrenme problemidir. Etiket kategorik olduğu için de bir sınıflandırma problemidir.

Kredi başvurusu örneği: İnsanlar kredi kartı başvurusu yapıyor. Yaş, evlilik durumu, yıllık maaş gibi öz nitelikleri var. Önceki başvuranların bilgileri ve hangisinin kabul, hangisinin ret edildiği elimizde. Bu etiketli veriden bir öğrenme modeli kurulur ve yeni başvuru bu modele göre değerlendirilir.

Her sınıflandırma probleminde verinin k tane özniteliği (attribute) ve her veri örneğinin (data instance) bir etiketi bulunur. Slayttaki tabloda 15 veri örneği, 4 öznitelik ve bir etiket vardı.

Bu gerçekleşmiş veri kümesi üzerinden gözetimli öğrenme algoritmasıyla bir model kurulur. Buna sınıflandırma modeli (classification model) de denir. Model, yeni gelen verinin etiketini tahmin etmek için kullanılır.

Bu algoritmalara "supervised" denmesinin sebebi, etiketli verinin modelin eğitimini denetlemesidir. Gözetimsiz algoritmalarda ise etiketli veri yoktur, yalnızca veri içindeki dağılım ve yapı yakalanmaya çalışılır.

7.2 Eğitim ve Test

Gözetimli öğrenmede iki aşama vardır: eğitim (training) ve test. Elimizdeki etiketli verinin belli bir oranı eğitim, kalanı test verisi olarak seçilir. Eğitim verisi %70-80 seçildiğinde kalan %30-20 test verisi olur.

Test verisinin de etiketi bilindiği için, eğitim verisinden öğrenilmiş modelin tahminleri gerçek etiketlerle karşılaştırılarak başarı ölçülür. Kullanılan ölçütlerden biri doğruluktur (accuracy):

$$\text{Accuracy} = \frac{\text{doğru sınıflandırılan test örneği sayısı}}{\text{toplam test örneği sayısı}}$$

Hoca, karışıklık matrisinin (confusion matrix) ilerideki derslerde görüleceğini söyledi. Bu matristen kesinlik (precision), duyarlılık (recall) ve F-skoru (F-score) gibi ölçütler de çıkarılır. Yani doğruluk, bir gözetimli öğrenme algoritmasının başarısını ölçen tek ölçüt değildir.

Eğitim verisinden öğrenmek şu demektir: Algoritma verilen veri kümesine uygulanır, görev (task T) gerçekleştirilir ve görevin ne kadar başarılı olduğu bir performans ölçütüyle belirlenir.

> **Hocanın vurgusu:** Öğrenmenin gerçekleşip gerçekleşmediğini mutlaka bir performans ölçütüne göre belirlemek gerekir.

7.3 Öğrenmeden Ne Kadar Başarı Elde Edilir?

Hoca şu düşünce deneyini yaptı: Kredi başvurusu probleminde iki etiket var, yes ve no. Hiçbir öğrenme yapmadan birini söylesem başarı %50 olur. Öğrenmeyle hedef bu %50'yi %80'lere, %90'lara taşımaktır. Aksi halde öğrenme için harcanan emeğin pek anlamı olmaz. %100 gerçekte olan bir durum değildir, ama %85-90 gibi kabul edilebilir oranlarda doğru tahmin hedeflenir.

> **Ders sonrası not:** "Hiç öğrenmeden %50" yalnız iki sınıfın eşit dağıldığı durumda doğrudur. Hocanın az sonra verdiği kredi örneğinde 15 kişinin 9'u yes, 6'sı no etiketlidir. Hep "yes" demek bile $9/15 = \%60$ doğruluk verir. Doğrusu büyük olasılıkla, karşılaştırma noktası olarak çoğunluk sınıfını tahmin etmenin başarısını almaktır.

7.4 Web Madenciliğinde Tahmin Edilen Etiketler

Hoca web verisi üzerinden tahmin edilen etiketlere şu örnekleri verdi:

- **E-ticaret:** Müşteri bu ürünü satın alacak mı?
- **Çevrimiçi oyun:** İngilizce veya başka bir yabancı dil öğreten oyunlarda, tıklama akışına bakılarak öğrencinin sıkılıp sıkılmadığı, oyunu bırakıp bırakmayacağı.
- **Bankacılık veya alım satım (trading) platformu:** Tıklama akışına bakılarak anomali, yani dolandırıcılık (fraud) olup olmadığı.

Bunlar ikili (binary) tahminlerdir.

7.5 Eğitim ve Test Dağılımı

Öğrenmenin başarısını doğru test edebilmek için eğitim ve test verisinin dağılımları benzer olmalıdır. Toplam 100 veriniz olsun ve bunun %80'ini, yani 80 tanesini eğitim, %20'sini, yani 20 tanesini test verisi seçin. Bu iki parçanın dağılımı benzer olmalıdır.

> **Hocanın vurgusu:** Test verisinin dağılımı tamamen farklıysa, öğrenmeyi başarılı şekilde yapmış olsanız bile test sonuçlarınız başarısız çıkar.

8. Karar Ağacı

8.1 Veri ve Ağacın Yapısı

Kredi başvurusu (loan application) örneğine dönüldü: 15 veri örneği, 4 öznitelik ve bir sınıf. 9 kişiye evet, 6 kişiye hayır denmiştir.

Karar ağacı (decision tree) algoritmasında model bir ağaç veri yapısıdır. İki tür düğüm vardır:

- **Karar düğümleri (decision nodes):** Kök ve ara düğümler. Her biri veri içindeki bir özniteliktir.
- **Yaprak düğümleri (leaf nodes):** Ağacın en altındaki düğümler. Veri örnekleri buralara düşer ve yaprak bir sınıf etiketi taşır.

8.2 Ağacın Kurulması

Ağaç kökten başlayarak kurulur:

1. Bütün veri düşünülür ve veriyi en iyi ayrıştıracak öznitelik bir ölçüte göre seçilir. Dört aday içinden örnekte Age seçilmiştir.
2. Veri, özniteliğin değerlerine göre dallara dağıtılır. Age için üç değer vardır: young, middle ve old.
3. Her dalda kalan üç öznitelik içinden o dalı en iyi ayrıştıran seçilir. Örnekte young dalında has_job, middle dalında own_house, old dalında credit_rating çıkmıştır.
4. Veri örnekleri bu özniteliklerin değerlerine göre yapraklara dağıtılır. Örneğin Age = young olan ilk 5 örnek, has_job yanıtına göre iki dala ayrılır. credit_rating'in üç değeri olduğu için onun üç alt dalı vardır.
5. Her yaprağa düşen örneklerin çoğunluk (majority) etiketi yaprağın sınıfı olur.

Örnekteki ağaçta veri öyle dağılmıştır ki yanlış sınıfa düşmüş hiçbir örnek yoktur. Yani ağaç, eğitim verisini yapraklara çok iyi dağıtmıştır.

Yeni bir veri geldiğinde ağaç yukarıdan aşağı izlenir. Hocanın örneğinde yeni verinin Age yanıtı young'dır, sonraki düğümde has_job yanıtı false'tur. Düştüğü yaprağın sınıfı no olduğu için kredi başvurusu reddedilir.

Hoca, karar ağacı algoritmasının Algoritma Analizi dersinden hatırlanacak böl ve yönet (divide and conquer) tabanlı bir algoritma olduğunu söyledi. Sözde kodun ayrıntısına girmedi.

8.3 Entropi ve Bilgi Kazancı

Düğümler seçilirken kullanılan ölçütler bilgi kazancı (information gain) ve kazanç oranıdır (gain ratio). Bilgi kazancı, içinde entropi ölçütünü kullanır. Entropi belirsizliği ifade eder: Belirsizlik varsa değeri yüksektir. Hoca iki benzetme yaptı:

- **Torba ve bilyeler:** Bir torbaya yes ve no etiketli bilyeler attığınızı düşünün. Elinizi sokup bir avuç bilye çıkardığınızda hepsi yes ise belirsizlik yoktur ve entropi düşüktür. Avucunuza hem yes hem no geliyorsa belirsizlik vardır ve entropi yüksektir.
- **Telefon:** Sizi biri arıyor. Telefonu açtığınızda karşınızda kimin olacağını biliyorsanız belirsizlik yoktur ve entropi sıfırdır. Karşınızdaki kişi sürprizse belirsizlik vardır ve entropi yüksektir.

Hoca ardından bilgi kazancını şöyle bağladı: Age'in bütün değerleri aynı olsaydı belirsizlik olmazdı ve bilgi kazancı düşük olurdu. Oysa örnekte Age'in yaklaşık üçte biri young, üçte biri middle, üçte biri old'dur. 15 bilyeyi bu değerlerle etiketleyip torbaya atsanız avucunuza ne geleceğini öngöremezsiniz. Bu yüzden hocaya göre Age'in entropisi, dolayısıyla bilgi kazancı ve ayrıştırıcı gücü yüksektir. Ağaç, en yüksek bilgi kazancına sahip öznitelikten başlanarak kurulur. Hoca örnekteki ağacı Age, sonra soldan sağa has_job, own_house ve credit_rating olarak bu şekilde açıkladı.

> **Ders sonrası not:** Hoca "Age'in değerleri dengeli dağıldığı için entropisi yüksek, bu yüzden bilgi kazancı yüksek" dedi; doğrusu büyük olasılıkla şudur: Bilgi kazancı, özniteliğin kendi değerlerinin entropisine değil, bölmeden sonra **sınıf etiketinin**

entropisindeki azalmaya bakar:

$$\text{gain}(D, A) = \text{entropy}(D) - \sum_j \frac{|D_j|}{|D|} \text{entropy}(D_j)$$

Hocanın aşağıda verdiği dal sayılarıyla hesap: entropy$(D) = 0{,}971$ bit. Age için kalan entropi $0{,}888$, kazanç $0{,}083$'tür. own_house için kalan entropi $0{,}551$, kazanç $0{,}420$'dir. Age'in değer dağılımının entropisi ($1{,}585$) own_house'unkinden ($0{,}971$) yüksek olduğu halde kazancı daha düşüktür. Yani bu ölçüte göre kök Age değil own_house olmalıdır. Yüksek bilgi kazancı "belirsizliği yüksek öznitelik" değil, dalları sınıf açısından en saf yapan öznitelik demektir.

Ek bilgi (derste anlatılmadı): Hoca kazanç oranını yalnız adıyla andı. Bu ölçüt, bilgi kazancını özniteliğin değer dağılımının entropisine böler. Çünkü bilgi kazancı çok sayıda farklı değeri olan öznitelikleri kayırır. Bölme sayesinde bu eğilim dengelenir.

8.4 Basit Hata Sayımı ile Karşılaştırma

Hoca, bilgi kazancından farklı bir bakış olarak iki özniteliği yanlış sınıflanan örnek sayısıyla karşılaştırdı. Her dalın etiketi çoğunlukla belirlenir ve o etikete uymayan örnekler sayılır.

Age ile bölünce:

Dal	Yes	No	Çoğunluk	Hata
young	2	3	no	2
middle	3	2	yes	2
old	4	1	yes	1
Toplam	9	6		**5**

Hoca toplamı "2, 4, 5" diye saydı. old dalı için yes ve no sayılarını söylemedi. Bunlar toplam 9 yes ve 6 no'dan bulunur (ara adım eklendi).

own_house ile bölünce:

Dal	Yes	No	Çoğunluk	Hata
true	6	0	yes	0
false	3	6	no	3
Toplam	9	6		**3**

Hoca her dalın yalnız hata sayısını söyledi: true dalında 0, false dalında 3. Yes ve no sayıları toplam 9 yes ve 6 no'dan bulunur (ara adım eklendi).

Bu bakışla own_house, 3 hatayla Age'in 5 hatasından daha başarılıdır.

Hocanın buradan çıkardığı ders daha geneldir. Aynı veri üzerinde birden fazla ağaç kurulabilir. Düğüm seçme ölçütü değişince ağacın yapısı, ağaç model olduğu için de tahmin başarısı değişir. Bu ölçüt, algoritmanın parametrelerinden biridir.

> **Hocanın vurgusu:** Aynı algoritmayı kullansanız bile parametreler değişince sonuç değişir. Bir algoritmayı bir veri kümesinde çalıştırırken parametreleri sürekli değiştirerek en başarılı kullanımını araştırmak gerekir.

9. Kural Tabanlı Sınıflandırma ve Aşırı Öğrenme

9.1 Ağaçtan veya Doğrudan Veriden Kural

Bir karar ağacından kurallar çıkarılabilir. Hocanın gösterdiği ağaç üç kurala dönüştürülmüştü:

- own_house = true ise sınıf yes.

- own_house = false ve has_job = true ise sınıf yes.
- Diğer durumda sınıf no.

Ağaç kurmadan da kural çıkarılabilir. Önceki derste öğrenilen birliktelik kuralı madenciliği, burada sınıf birliktelik kuralı madenciliği (class association rule mining) biçiminde kullanılır. Apriori gibi algoritmalarla kurallar doğrudan veriden çıkarılır. Minimum destek ve minimum güven (confidence) değerleri kullanıcı tarafından verilir. Karar ağacında model ağaç yapısıydı. Burada model kural listesidir.

9.2 Kural Listesiyle Tahmin

Yeni verinin etiketi tahmin edilirken kuralların hangi sırayla uygulanacağı belirlenmelidir, çünkü bazı kurallar kendisinden sonra gelenlerden daha genel olabilir. Kurallar destek ve güven değerlerine göre sıralanır. Sonra:

1. Sıralı listedeki birinci kurala gidilir. Örnekte kural şudur: Age = young ve has_job = true ise sınıf yes.
2. Kural yeni veriyi etiketleyebiliyorsa etiketleme biter.
3. Etiketleyemiyorsa ikinci kurala, sonra üçüncüye geçilir.
4. Hiçbir kural etiketleyemiyorsa varsayılan (default) sınıflardan biri atanır. Kredi örneğinde bu yes veya no'dur.

   Ders sonrası not: Hoca kuralların destek değerine göre sıralandığını söyledi; doğrusu büyük olasılıkla, CBA yönteminde kuralların önce güven, eşitse destek değerine göre sıralanmasıdır.

9.3 Ezberleme Sorusu

Soru: Bu model ezberlemeye götürmez mi? Örneğin eğitim verisinde maaş ve yaşa bakıp "evet" denmiş olabilir, ama kredi notu düşük olduğu için "hayır" da denebilirdi. İlk uyan kuralın direkt evet demesi modeli yanlış öğretmez mi, test sonucunu kötü çıkarmaz mı?

Cevap: Hoca "doğru, haklısın" dedi. Önce listenin mantığını açıkladı: Bir kuralın başta olmasının sebebi, verideki çok sayıda müşteride tekrarlanmış olmasıdır. Hoca, slaytta örnek olarak gösterilen sıranın desteğe göre olmadığını, en yüksek desteğe sahip 5/15 değerli kuralla başlanması gerektiğini söyledi. Çok müşteride tekrar etmiş kural muhtemelen yanıltmaz. Ama sonuçta kurallar yalnız eldeki veriden çıkarılır.

Bu durumun adı aşırı öğrenmedir (overfitting). Etiketli veriye çok bağlı öğrenme yapılırsa öğrenme sınırlı kalır ve etiketlemeler de ezbere dayanır. Hocaya göre kural tabanlı yöntemler bu yüzden diğer sınıflandırma yöntemlerine göre çok kullanılmıyor.

Karar ağacıyla karşılaştırması şöyledir: Karar ağacında veriyi en iyi tanımlayan ana kurallar çıkarılır. Sınıf birliktelik kurallarında ise minimum destek ve güven düşük belirlendiği için çok daha fazla ve ayrıntılı kural çıkar. Veri daha ayrıntılı öğrenildiği için ezberleme noktasına gidilir. Bu yüzden hocaya göre karar ağacındaki ezberleme problemi daha azdır.

> **Ders sonrası not:** Karşılaştırma yalnız bu iki yöntem için söylendi. Büyük olasılıkla karar ağacı da budanmazsa ezberlemeye açıktır. Hocanın gösterdiği ağaç da 15 eğitim örneğinin hepsini doğru sınıflandırıyordu.

10. Naive Bayes

Naive Bayes istatistiksel analize dayalı bir sınıflandırıcıdır. Örnekte insanlar bilgisayar alıyor ya da almıyor (buys_computer). 4 öznitelik ve 14 etiketli örnek var. 14 örnekten 9'u yes, 5'i no. Amaç, verilen bir X örneğinin etiketini tahmin etmektir.

Hoca önce modelleri hatırlattı: Karar ağacında model ağaçtı, sınıf birliktelik kurallarında kural listesiydi. Burada ise istatistiksel analizle bir model kurulacak.

10.1 Formül

İki sınıf olduğu için iki olasılık hesaplanır: X'in yes sınıfına ve no sınıfına ait olma olasılığı. Hangisi büyükse X o sınıfa atanır. Formül Bayes teoremidir:

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

İki denklemin paydası da $P(X)$ olduğu için karşılaştırılmaz. Yalnız paylara bakılır. Payda iki değer var:

- $P(C)$: Sınıfın veride görülme olasılığı. $P(\text{yes}) = 9/14$, $P(\text{no}) = 5/14$.
- $P(X \mid C)$: Verilen sınıf içinde X'in görülme olasılığı.

10.2 $P(X \mid C)$'nin Hesabı

$P(X \mid C)$ bulunurken X'in her özniteliğinin o sınıftaki görülme olasılığı ayrı ayrı bulunur ve bunlar çarpılır:

$$P(X \mid C) = \prod_j P(x_j \mid C)$$

Hoca yes sınıfı için ilk iki çarpanı tablodan saydı:

- Birinci öznitelik: Yes olan 9 örnekte age ≤ 30 olanlar 2 tanedir. $P(\text{age} \leq 30 \mid \text{yes}) = 2/9$.
- İkinci öznitelik: Yes olan 9 örnekte income = medium olanlar 4 tanedir. $P(\text{income} = \text{medium} \mid \text{yes}) = 4/9$.

Diğer öznitelikler için de aynısı yapılır ve hepsi çarpılınca $P(X \mid \text{yes})$ bulunur.

> **Ek bilgi (derste anlatılmadı):** Çarpım, özniteliklerin sınıf verildiğinde birbirinden koşullu bağımsız olduğu varsayımına dayanır. Yöntemin "naive" (saf) diye anılması bu varsayımdan gelir. Varsayım gerçekte çoğu zaman tam sağlanmaz.

10.3 Sonuç

Karşılaştırılan iki değer şunlardır:

$$P(X \mid \text{yes}) \cdot \frac{9}{14} \qquad \text{ve} \qquad P(X \mid \text{no}) \cdot \frac{5}{14}$$

no sınıfı için de aynı yol izlenir: X'in her özniteliğinin no sınıfındaki olasılıkları çarpılır, sonuç $5/14$ ile çarpılır. Örnekte birinci değer daha büyük çıktığı için X yes sınıfına atanmıştır.

> **Ders sonrası not:** Hoca bulunan çarpımı "X'in yes sınıfına ait olma olasılığı" diye adlandırdı. $P(X)$'e bölünmediği için doğrusu, bu değerin yalnız o olasılıkla orantılı olmasıdır. Karşılaştırma için yeterlidir, ama kendisi bir olasılık değeridir denemez.

Bu sınıflandırıcıda model, hesaplanan olasılık değerleri ve Bayes teoremi formülleridir.

11. K En Yakın Komşu

K en yakın komşu (k-nearest neighbor) sınıflandırıcısında, iki boyutlu uzayda gösterilen etiketli bir veri düşünülür. Etiketler farklı renklerle gösterilmiştir. Yeni veri geldiğinde etiketi şöyle tahmin edilir:

1. k değeri algoritmaya en başta verilir. Örnekte $k = 6$'dır.
2. Yeni veriyle veri kümesindeki bütün veriler arasındaki uzaklık hesaplanır. Noktaların x_1 ve x_2 sayısal değerleriyle ifade edildiği varsayılırsa Öklid uzaklığı (Euclidean distance) kullanılabilir. Farklı uzaklık ölçütleri de kullanılabilir.
3. En yakın k örnek seçilir.
4. Bu k örnek içinde en çok tekrar eden etiket yeni verinin etiketi olur. Örnekte çoğunluk "government" etiketindedir, dolayısıyla yeni veri government olarak etiketlenir.

Sözde kodun girdileri veri kümesi, k değeri ve tahmin edilecek veridir.

Bu sınıflandırıcının diğerlerinden farkı model kurmamasıdır. Karar ağacı, kural listesi ve Naive Bayes'te model veya olasılık değerleri çıkarıldıktan sonra yeni veri için tekrar veriye gidilmez. K en yakın komşuda ise her etiketlemede mutlaka verinin kendisine gitmek gerekir.

12. Test Verisi Seçimi ve Çapraz Doğrulama

Soru: Model kurulurken test verisini olabildiğince farklı çeşitlerden seçmenin bir yöntemi veya yaklaşımı var mı?

Cevap: Hoca öğrenme modelinin başarısının farklı şekillerde ölçülebileceğini söyledi ve yöntemleri sıraladı.

- **Oranla ayırma:** Veri %70'e %30 veya %80'e %20 ayrılır.
- **N katlı çapraz doğrulama (n-fold cross-validation):** Aşağıda açıklanıyor.
- **Rastgele seçim:** %30'luk test verisi belli bir noktadan sonraki veri olarak alınmak yerine, verinin içinden rastgele (random) elemanlar seçilir.

Hoca çapraz doğrulamayı bir tablo üzerinde gösterdi. Veri 8 parçaya (segment) ayrılsın:

1. Birinci parça test verisi, kalan parçalar eğitim verisi olarak alınır. Öğrenme yapılır, test verisinde tahmin yapılır ve birinci doğruluk değeri bulunur.
2. İkinci parça test verisi, kalanlar eğitim verisi olur. İkinci doğruluk değeri bulunur.
3. Bu işlem bütün parçalar için yapılır. 8 farklı doğruluk değeri bulunur ve ortalaması alınır.

Hoca sonunda on katlı (ten-fold) çapraz doğrulamayı da seçenek olarak andı.

Yöntem ne olursa olsun dağılıma bakmak gerekir. Veri normal dağılım eğrisine göre dağılmışsa, test verisi de bu dağılımı yansıtmalıdır. Eğitim verisi alfa, gama veya beta dağılımındaysa test verisi de aynı dağılımı yansıtmalıdır.

> **Hocanın vurgusu:** Sonuçlar çok başarılı veya tamamen başarısız çıkarsa bakılması gerekenlerden biri test verisidir. Çok başarılı olmanın altında test verisinin çeşitlilik içermemesi ve önceki öğrenmeyi yalnızca yansıtması yatıyor olabilir. Eğitim ve test verisinin dağılımına mutlaka ayrıca bakılmalıdır.

Dersin Toparlaması

Dersin ilk yarısında, birliktelik kurallarının göz ardı ettiği sıra bilgisini kullanan sıralı örüntü madenciliği işlendi. Size ve length ayrımı, alt sequence tanımı, müşteri bazlı veri formatı ve %25 destekli örnek görüldü. GSP'nin Apriori mantığını sequence'lara taşıdığı anlatıldı. İkinci yarıda makine öğrenmesi algoritmalarının genel haritası çizildi, gözetimli öğrenmenin eğitim, test ve başarı ölçümü aşamaları hatırlatıldı. Karar ağacı, sınıf birliktelik kuralları, Naive Bayes ve k en yakın komşu sınıflandırıcıları üzerinden aynı problemin farklı model türleriyle nasıl çözüldüğü gösterildi. Aşırı öğrenme ve test verisinin temsil gücü, bu yöntemlerin ortak değerlendirme konusu oldu.

Gelecek hafta gözetimsiz öğrenme tarafından kümeleme algoritmaları hızlıca işlenecek, sonra bilgi erişimi (information retrieval) konusuna girilecek. Regresyon ve PCA'nın ayrıntısına girilmeyecek. Ardından makine öğrenmesi algoritmalarının web madenciliğindeki kullanımlarına geçilecek: önce web içerik madenciliği (web content mining), sonra web link analizi ve web yapı madenciliği (web structure mining), son olarak web kullanım madenciliği (web usage mining).

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.